MovieLens Recommendation System

HarvardX Data Science Professional Certificate: PH125.9x

Fidan Alasgarova

7 March 2026

Introduction

Before machine learning, computers had to be explicitly programmed to perform every task. Unlike traditional programming, machine learning algorithms solve problems by observing solutions, discovering methods independently and processing data into naturally intuitive solutions enabling applications such as recommendation systems. Recommendation systems predict content that might interest users. The system learns patterns from examples and applies them to new situations. Recommendation systems improve user experience, provide more relevant suggestions, and drive revenue: personalization can reduce customer acquisition costs by as much as 50%, lift revenues by 5–15%, and increase marketing ROI by 10–30% (McKinsey, 2023). These benefits illustrate why personalization and recommendation engines are strategically valuable. Collaborative filtering predicts user preferences based on similarities between users, assuming those with similar tastes will enjoy similar items. Linear regression–based collaborative filtering offers a practical balance of accuracy and computational efficiency. Matrix factorization is a technique commonly used in recommender systems that models latent (hidden) factors representing user attitudes and product characteristics to better predict preferences.

Project Overview

In 2006, Netflix launched a data science competition offering a $1 million reward to enhance its recommendation algorithm by improving prediction accuracy by 10%. The winners were announced in 2009. Although Netflix never fully implemented the winning algorithm, the challenge highlighted the importance of collaborative filtering techniques and provided a significant boost to their development. This project builds on the same concept of developing a movie recommendation system using the MovieLens dataset, which contains 10 million user ratings. The dataset is divided into two subsets: a training set (edx) used to construct and tune the model, and a final hold-out test set used to assess predictive accuracy of the final model.

```
#############################################################
# Create edx and final_holdout_test sets
#############################################################

# Note: this process could take a couple of minutes

if(!require(tidyverse)) install.packages("tidyverse", repos = "http://cran.us
.r-project.org")
if(!require(caret)) install.packages("caret", repos = "http://cran.us.r-proje
ct.org")

library(tidyverse)
library(caret)

# MovieLens 10M dataset:
# https://grouplens.org/datasets/movielens/10m/
# http://files.grouplens.org/datasets/movielens/ml-10m.zip

options(timeout = 600)

dl <- "ml-10M100K.zip"
if(!file.exists(dl))
  download.file("https://files.grouplens.org/datasets/movielens/ml-10m.zip",
dl)

ratings_file <- "ml-10M100K/ratings.dat"
if(!file.exists(ratings_file))
  unzip(dl, ratings_file)

movies_file <- "ml-10M100K/movies.dat"
if(!file.exists(movies_file))
  unzip(dl, movies_file)

ratings <- as.data.frame(str_split(read_lines(ratings_file), fixed("::"), sim
plify = TRUE),
                         stringsAsFactors = FALSE)
colnames(ratings) <- c("userId", "movieId", "rating", "timestamp")
ratings <- ratings %>%
  mutate(userId = as.integer(userId),
         movieId = as.integer(movieId),
         rating = as.numeric(rating),
         timestamp = as.integer(timestamp))

movies <- as.data.frame(str_split(read_lines(movies_file), fixed("::"), simpl
ify = TRUE),
                        stringsAsFactors = FALSE)
colnames(movies) <- c("movieId", "title", "genres")
movies <- movies %>%
```

```
  mutate(movieId = as.integer(movieId))

movielens <- left_join(ratings, movies, by = "movieId")

# Final hold-out test set will be 10% of MovieLens data
set.seed(1, sample.kind="Rounding") # if using R 3.6 or later
# set.seed(1) # if using R 3.5 or earlier
test_index <- createDataPartition(y = movielens$rating, times = 1, p = 0.1, l
ist = FALSE)
edx <- movielens[-test_index,]
temp <- movielens[test_index,]

# Make sure userId and movieId in final hold-out test set are also in edx set
final_holdout_test <- temp %>%
  semi_join(edx, by = "movieId") %>%
  semi_join(edx, by = "userId")

# Add rows removed from final hold-out test set back into edx set
removed <- anti_join(temp, final_holdout_test)
edx <- rbind(edx, removed)

rm(dl, ratings, movies, test_index, temp, movielens, removed)
```

The primary goal is to create an algorithm capable of predicting ratings with a Root Mean Square Error (RMSE) lower than 0.86490. As more advanced methods can be challenging in implementation due to hardware constraints, regularized linear regression-based bias model is considered a suitable choice for the Movielens dataset. The performance of recommendation algorithms is limited by data quality: sparse or biased data produce unreliable results, and increasing reliance on personal data raises important privacy concerns. To improve bias based recommendations system, data needs to be regularized, shrinking extreme estimates for users or items with limited data to reduce variance and prevent overfitting. Additionally, matrix factorization model was used to improve performance, account for latent factors and reduce RMSE. As some users prefer certain genres or may rate higher or lower than average, the matrix factorization model can observe these trends. It takes into account latent factors of user preferences and behaviors and product characteristics to predict ratings for similar items based on these clues.

Data Exploratory Analysis

The quality of data directly impacts the performance of machine learning algorithms. This report begins with an exploratory analysis of the Movielens dataset to understand its structure and extract insights. The Movielens dataset is divided into an edx set comprising of 9,000,055 entries and a validation set (final_holdout_set) with 999,999 entries, reflecting a 90/10 partition.

```
# Number of ratings in the edx and final_houldout_test datasets
dim(edx)

## [1] 9000055       6

dim(final_holdout_test)

## [1] 999999      6
```

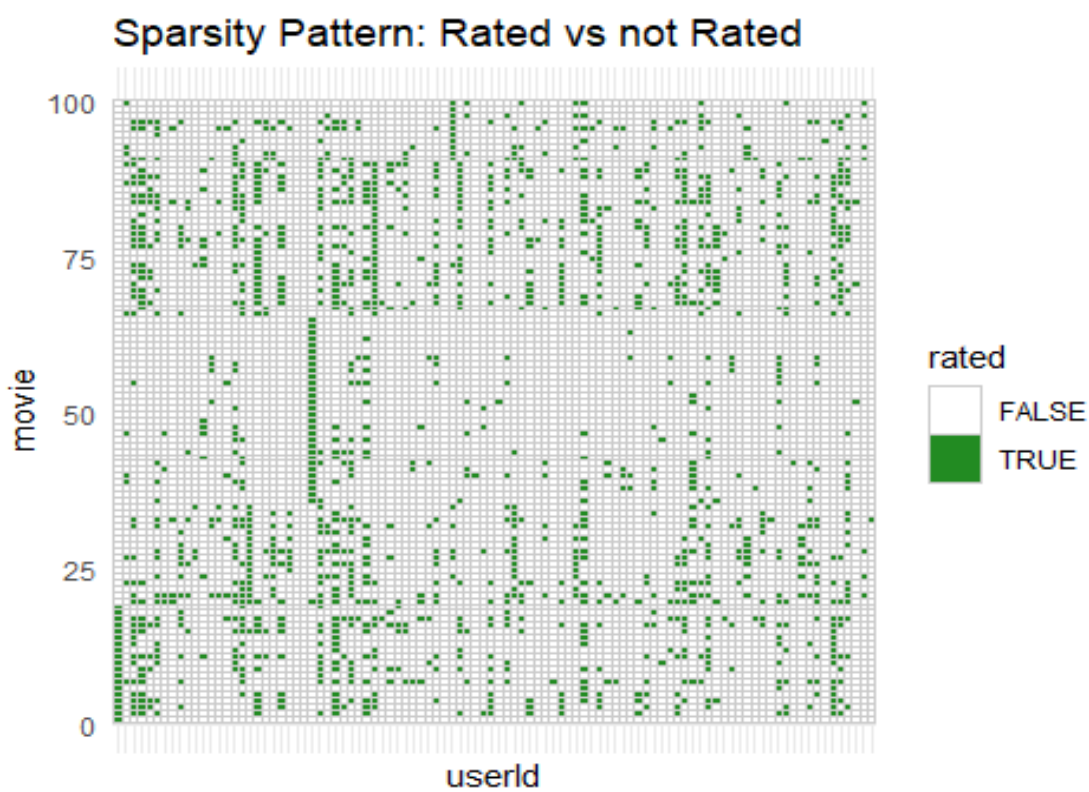
The edx set includes 69,878 unique users and 10,677 unique movies, resulting in 746,087,406 possible user–movie rating combinations (if every user rated every movie), but only 9,000,055 actual ratings.

```
# Number of unique users and unique movies, and max possible number of rating
in the edx dataset
length(unique(edx$userId))

## [1] 69878

length(unique(edx$movieId))

## [1] 10677

length(unique(edx$userId)) * length(unique(edx$movieId))

## [1] 746087406
```

A sample sparsity matrix of 100 movies by 100 users is presented, with green squares indicating available user–movie ratings.

```
# Build rating matrix
library(Matrix)
library(tidyr)
library(dplyr)

rating_matrix <- edx %>%
  select(userId, movieId, rating, title) %>%
  pivot_wider(
    id_cols = c(movieId, title),
    names_from = userId,
    values_from = rating,
    values_fill = 0)
```

```r
matrix_subset <- rating_matrix[1:100, 1:100]
matrix_subset <- matrix_subset[, -c(1, 2)]
matrix_subset <- matrix_subset %>%
  as.data.frame() %>%
  mutate(movie = row_number()) %>%
  pivot_longer(
    cols = -movie,
    names_to = "userId",
    values_to = "rating")

# Visualize sparsity matrix
matrix_subset %>%
  mutate(rated = rating > 0) %>%
  ggplot(aes(x = userId, y = movie, fill = rated)) +
  geom_tile(color="grey80") +
  scale_fill_manual(values = c("white", "#228B22")) +
  labs(title = "Sparsity Pattern: Rated vs not Rated") +
  theme_minimal() +
  theme(
    axis.text.x  = element_blank())
```



The dataset is very sparse, meaning most movie ratings are missing and most users only rate a small number of movies. The recommendation algorithm will try to fill those missing values and predict as accurately as possible the ratings a user would give to a movie based on existing ratings.

Each sample (row) in the dataset represents a movie rated by a user, uniquely identified by the numeric attributes movieId and userId. The edx dataset contains no duplicate observations, and no user has rated the same movie more than once.

```
# Number of duplicate rows in the edx dataset
sum(duplicated(edx))

## [1] 0

# Number of duplicate ratings in the edx dataset
edx %>%
  count(userId, movieId) %>%
  filter(n > 1)

## [1] userId  movieId n
## <0 rows> (or 0-length row.names)
```

Ratings are numeric values ranging from 0.5 (lowest) to 5 (highest), in increments of 0.5.

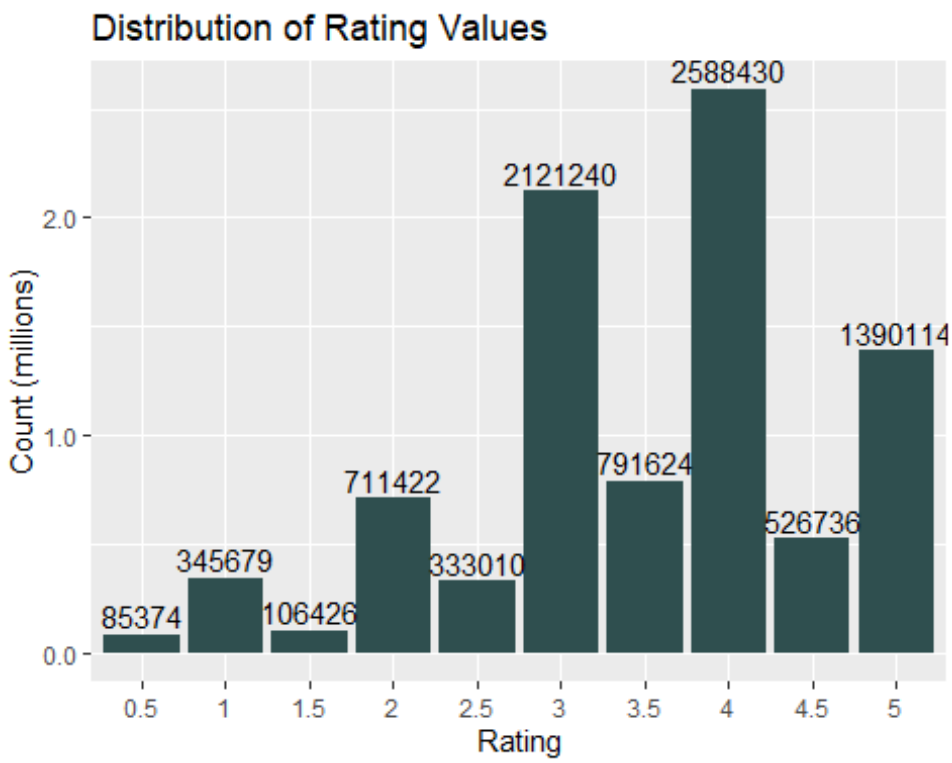
```
# Unique rating values in the edx dataset
sort(unique(edx$rating))

##  [1] 0.5 1.0 1.5 2.0 2.5 3.0 3.5 4.0 4.5 5.0
```

The average rating for the validation set is 3.51

```
# Global mean in the edx dataset
mu <- mean(edx$rating)
mu

## [1] 3.512465
```

The total ratings distribution indicates that the most frequent rating for all movies is 4. Additionally, the rating distribution is uneven, reflecting the varying popularity of different movies among user.

```
# Distribution of rating values (05. to 5) in the edx dataset
rating_distribution <- as.data.frame(table(edx$rating))
colnames(rating_distribution) <- c("Rating", "Count")

ggplot(rating_distribution, aes(x = Rating, y = Count))+
  geom_bar(stat = "identity", fill = "darkslategrey") +
  geom_text(
    aes(label = Count), vjust = -0.3)+
  scale_y_continuous(
    labels = function(x) sprintf("%.1f", x / 1e6))+
  labs(y = "Count (millions)") +
  labs(title = "Distribution of Rating Values")
```



The glimpse into the edx set reveals that two columns contain combined values: the “title” column merges the movie title with its year of release, and the “genres” column includes multiple genres associated with the movie. Additionally, values in the timestamp column are stored in Unix time format.

```
# Glimpse into edx dataset
head(edx)

##   userId movieId rating timestamp                         title
## 1      1     122      5 838985046              Boomerang (1992)
## 2      1     185      5 838983525               Net, The (1995)
## 4      1     292      5 838983421               Outbreak (1995)
## 5      1     316      5 838983392               Stargate (1994)
## 6      1     329      5 838983392 Star Trek: Generations (1994)
## 7      1     355      5 838984474       Flintstones, The (1994)
##                          genres
## 1                Comedy|Romance
## 2          Action|Crime|Thriller
## 4   Action|Drama|Sci-Fi|Thriller
## 5        Action|Adventure|Sci-Fi
## 6 Action|Adventure|Drama|Sci-Fi
## 7        Children|Comedy|Fantasy
```

To account for release-year effects, the title column is split into separate title and year variables. The genres column is simplified by retaining only the first listed genre as the primary genre, enabling clearer estimation of genre-specific effects. The timestamp

variable is translated to a “review” column with yearly format to capture trends in rating behavior. Both the edx and validation sets will be adjusted accordingly.

```
# Rename edx and final_houldout set
edx_set <- edx
validation_set <- final_holdout_test

# Separate title and year columns in the edx set
edx_set <- edx_set %>%
  mutate(year = as.numeric(str_extract(title, "\\(\\d{4}\\)") %>%
                              str_remove_all("[()]")),
         title = str_remove(title, "\\s*\\(\\d{4}\\)"))

# Split genre column to keep only the first value in the edx set
edx_set <- edx_set %>%
  mutate(
    genre = str_extract(genres, "^[^|]+")) %>%
  select(-genres)

# Convert timestamp into date format in the edx set
library(lubridate)
edx_set <- edx_set %>%
  mutate(review = year(as_datetime(timestamp)))

# Split title and year column in the validation set
validation_set <- validation_set%>%
  mutate(year = as.numeric(str_extract(title, "\\(\\d{4}\\)") %>%
                              str_remove_all("[()]")), title = str_remove(titl
e, "\\s*\\(\\d{4}\\)"))

# Split genre column to keep only the first value in the validation set
validation_set <- validation_set %>%
  mutate(genre = str_extract(genres, "^[^|]+")) %>%
  select(-genres)

# Convert timestamp into date format in the validation set
validation_set <- validation_set %>%
  mutate(review = year(as_datetime(timestamp)))

# Glimpse into edx_set dataset
head(edx_set)

##   userId movieId rating timestamp              title    year  genre review
## 1      1     122       5 838985046              Boomerang 1992   Comedy 1996
## 2      1     185       5 838983525               Net, The 1995   Action 1996
## 4      1     292       5 838983421               Outbreak 1995   Action 1996
## 5      1     316       5 838983392               Stargate 1994   Action 1996
## 6      1     329       5 838983392 Star Trek: Generations 1994   Action 1996
## 7      1     355       5 838984474       Flintstones, The 1994 Children 1996
```

Summary: The MovieLens dataset is large but highly sparse, containing only a small fraction of all possible user–movie rating combinations, with no duplicate observations. The ratings range from 0.5 to 5.0, with an average rating of 3.51, and a rating of 4.0 being the most frequently given score. The title, timestamp, and genre columns were adjusted to enable data extraction for further analysis.

Methodology

A linear regression–based collaborative filtering model was implemented to predict ratings given to movies by users. The model assumes that each rating can be broken down into systematic components that reflect consistent patterns in the data. Specifically, some users rate higher or lower than average (user bias), some movies consistently receive higher or lower ratings (movie bias), certain genres attract different ratings (genre bias), movies released in different periods exhibit varying rating patterns (release year bias), and ratings may vary depending on when they were given (review year bias). The base model assumes the average across every user and every movie as all the predicted ratings. Where $Y_{u,i}$ is the predicted rating of user u and movie i and μ is the average rating across all entries.

$$Y_{u,i} = \mu + \epsilon_{u,i}$$

To enhance our model, we will incrementally add independent bias terms to account for movie bias b_i, user bias b_u, genre bias b_g, year of release bias b_y and review date bias b_r. The final model will include these adjustments.

$$Y_{u,i} = \mu + b_i + b_u + b_g + b_y + b_r + \epsilon_{u,i}$$

At each step while adding one more bias, the evaluation of the models will be based on Root Mean Square Error (RMSE) between the predicted ratings and actual ratings in the test set. In the formula below, $y(u,i)$ is the rating given by user u for movie i, and $\hat{y}(u,i)$ represents the predicted rating for movie i by user u. The RMSE can be calculated as follows:

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{u,i}\left(\hat{y}_{u,i} - y_{u,i}\right)^2}$$

The edx_set dataset is further split into training (80%) and test (20%) sets to train the model and assess the contribution of each bias term to predictive performance.

```r
# Create test set 80% and train set 20% from edx dataset
set.seed(1)
train_index <- createDataPartition(y = edx_set$rating,
                                   times = 1, p = 0.80, list = FALSE)

train <- edx_set[train_index, ]
test  <- edx_set[-train_index, ]
test <- test %>%
  semi_join(train, by = "movieId") %>%
  semi_join(train, by = "userId")
```

To prevent overfitting, regularization was incorporated to shrink extreme parameter estimates toward zero, thereby improving generalization. The regularization parameter was selected by minimizing RMSE in the edx test dataset.

$$\frac{1}{N}\sum_{u,i}\left(y_{u,i}-\mu-b_i-b_u-b_g-b_r-b_y\right)^2+\lambda\left(\sum_i b_i^2+\sum_u b_u^2+\sum_g b_g^2+\sum_r b_r^2+\sum_y b_y^2\right)$$

Furthermore, regularized bias model accounts for systematic shifts against average values, but it does not capture specific user-movie interactions, considering latent factors. As some users prefer certain genres or may rate higher or lower than average, the matrix factorization model can observe these trends and take them into account while the model makes rating predictions with better precision. The matrix factorization model reduces the need for including genre, year of release and year of review bias into the model, as it can capture similar patterns explicitly. While bias based model captures global structure, latent factors capture specific interaction effects. The matrix factorization model takes into account movie i and user u effects, while adding p_u (for latent user preferences) and q_i (for latent movie preferences) as low-dimensional latent vectors representing hidden user preferences and movie characteristics.

$$\hat{Y}_{u,i}=\mu+b_i+b_u+p_u^{\top}q_i$$

Both the regularized bias and matrix factorization methods are going to be used, to determine which method produces the lowest RMSE and achieve better predictions for movie ratings.

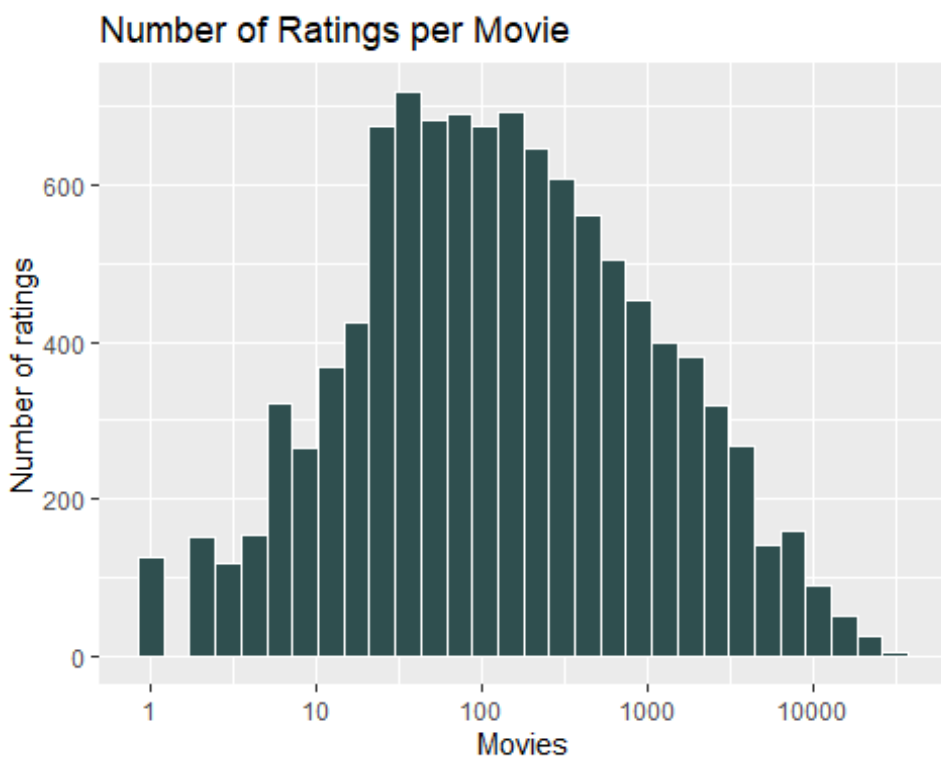
Analysis

The recommendation system will try to predict ratings for the movies while considering bias in data and achieving RMSE below 0.86490. The initial RMSE calculated from the average rating across the dataset is 1.060331.

```r
# RMSE based on global mean
RMSE_mu <- RMSE(edx_set$rating, mu)
RMSE_mu

## [1] 1.060331
```

The exploratory analysis reveals that ratings were not distributed evenly across all movies. Some films are more popular among viewers, resulting in higher average ratings for those titles. Further analysis shows considerable variation in the number of ratings each film has received. The impact of individual movies on the number of ratings is illustrated below.

```
# Number of ratings per movie in the edx dataset
edx_set %>%
  count(movieId) %>%
  ggplot(aes(n)) +
  geom_histogram( bins=30, fill = "darkslategrey", color = I("white")) +
  scale_x_log10() +
  labs(x = "Movies", y = "Number of ratings", title = "Number of Ratings per
Movie")
```



Given the disparity in ratings awarded to different movies, it is crucial to adjust for movie effect bias in the model. Additionally, from the exploratory analysis we saw that ratings were not assigned evenly across all movies, as some movies are more popular than other. The training algorithm was adjusted to account for movie bias b_i, to gradually improve the model:

$$Y_{u,i} = \mu + b_i + \epsilon_{u,i}$$

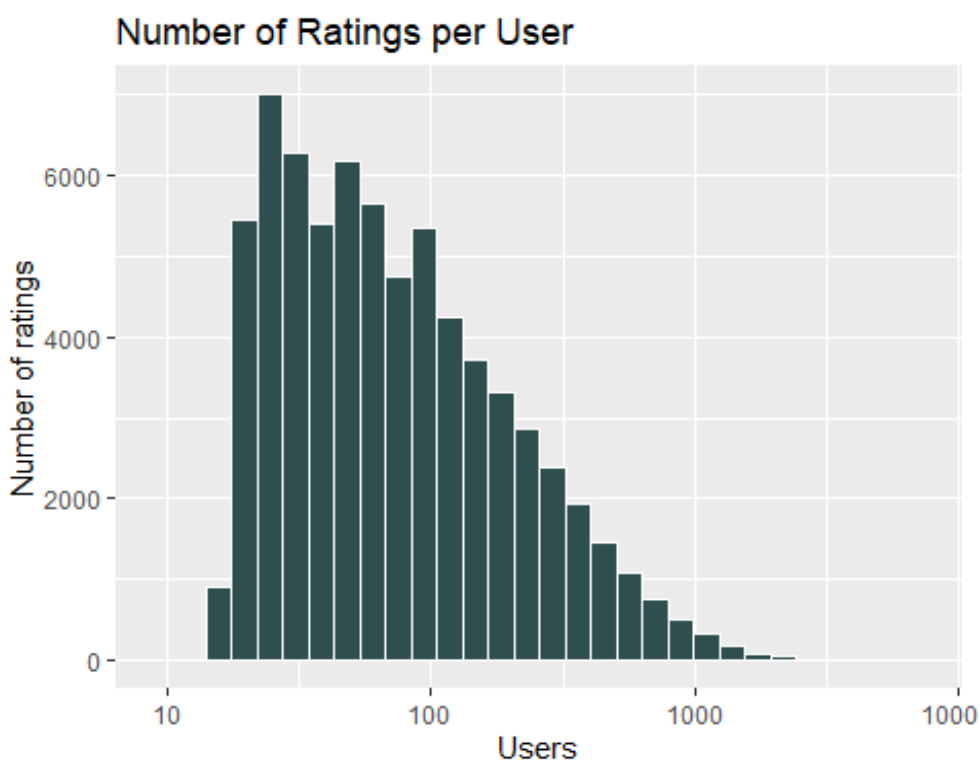
```
# Adjusting the model for movie effect b_i
b_i <- train %>%
  group_by(movieId) %>%
  summarize(b_i = mean(rating - mu))
```

```r
# RMSE based on mu, b_i
predicted_b_i <- test %>%
  left_join(b_i, by = "movieId") %>%
  mutate(pred = mu + b_i) %>%
  pull(pred)

RMSE(test$rating, predicted_b_i)

## [1] 0.943203
```

The calculated RMSE $\mu + b_i$: 0.943203

In the MovieLens dataset, some users have rated significantly more movies than others, while the majority of users have rated only a small number of movies.

```r
# Number of ratings per user in the edx dataset
edx_set %>% count(userId) %>%
  ggplot(aes(n)) +
  geom_histogram(bins = 30, fill = "darkslategrey", color = I("white")) +
  scale_x_log10() +
  labs(x = "Users", y = "Number of ratings", title = "Number of Ratings per U
ser")
```



The exploratory analysis revealed that the distribution of average ratings across users reflects systematic differences in rating behavior. While most users' average ratings center around the global mean, some users consistently give higher or lower ratings. Users

with very few ratings often exhibit more extreme averages, highlighting the need for regularization to stabilize user bias estimates.

To improve predictions, the model incorporates user bias terms alongside the overall average rating and movie bias.

$$Y_{u,i} = \mu + b_i + b_u + \epsilon_{u,i}$$

```
# Adjusting the model for user effect b_u
b_u <- train %>%
  left_join(b_i, by = "movieId") %>%
  group_by(userId) %>%
  summarize(b_u = mean(rating - mu - b_i))

# RMSE based on mu, b_i, b_u
predicted_b_u <- test %>%
  left_join(b_i, by = "movieId") %>%
  left_join(b_u, by = "userId") %>%
  mutate(pred = mu + b_i + b_u) %>%
  pull(pred)

RMSE(test$rating, predicted_b_u)

## [1] 0.8655254
```

The calculated RMSE $\mu + b_i + b_u$: 0.8655254

Genre related patterns are evident in the dataset. Rating frequency varies across genres, with some genres attracting significantly more user engagement - certain genres receive more ratings than others.

```
# Number of ratings per genre in the edx dataset
edx_set %>%
  group_by(genre) %>%
  summarize(ratings = n()) %>%
  arrange(desc(ratings))

## # A tibble: 20 × 2
##    genre              ratings
##    <chr>                <int>
##  1 Action             2560545
##  2 Comedy             2437260
##  3 Drama              1741668
##  4 Adventure           753650
##  5 Crime               529521
##  6 Horror              233074
##  7 Animation           218123
##  8 Children            181217
##  9 Thriller             94718
## 10 Documentary          80966
```

```
## 11 Sci-Fi                50254
## 12 Mystery               30536
## 13 Fantasy               26080
## 14 Musical               16264
## 15 Film-Noir             15811
## 16 Western               15300
## 17 Romance               12733
## 18 War                    2314
## 19 IMAX                     14
## 20 (no genres listed)        7
```

Average ratings differ systematically by genre, suggesting the presence of genre-specific preference effects – certain genres consistently achieve higher average ratings.

```
# Average rating per genre in the edx dataset
edx_set %>%
  group_by(genre) %>%
  summarize(
    ratings = n(),
    average = mean(rating)) %>%
  arrange(desc(average)) %>%
  mutate(average)
```

```
## # A tibble: 20 × 3
##    genre              ratings average
##    <chr>                <int>   <dbl>
##  1 Film-Noir            15811    4.15
##  2 Crime               529521    3.87
##  3 Documentary          80966    3.79
##  4 Mystery              30536    3.70
##  5 War                   2314    3.67
##  6 Drama              1741668    3.67
##  7 (no genres listed)       7    3.64
##  8 Musical              16264    3.64
##  9 Adventure           753650    3.56
## 10 Animation           218123    3.55
## 11 Western              15300    3.54
## 12 Thriller             94718    3.53
## 13 Comedy             2437260    3.45
## 14 Sci-Fi               50254    3.43
## 15 Action             2560545    3.42
## 16 Fantasy              26080    3.31
## 17 Romance              12733    3.28
## 18 Children            181217    3.25
## 19 Horror              233074    3.03
## 20 IMAX                    14    2.32
```

To improve predictive accuracy and reduce RMSE, the model accounts for these systematic differences by incorporating a genre bias term.

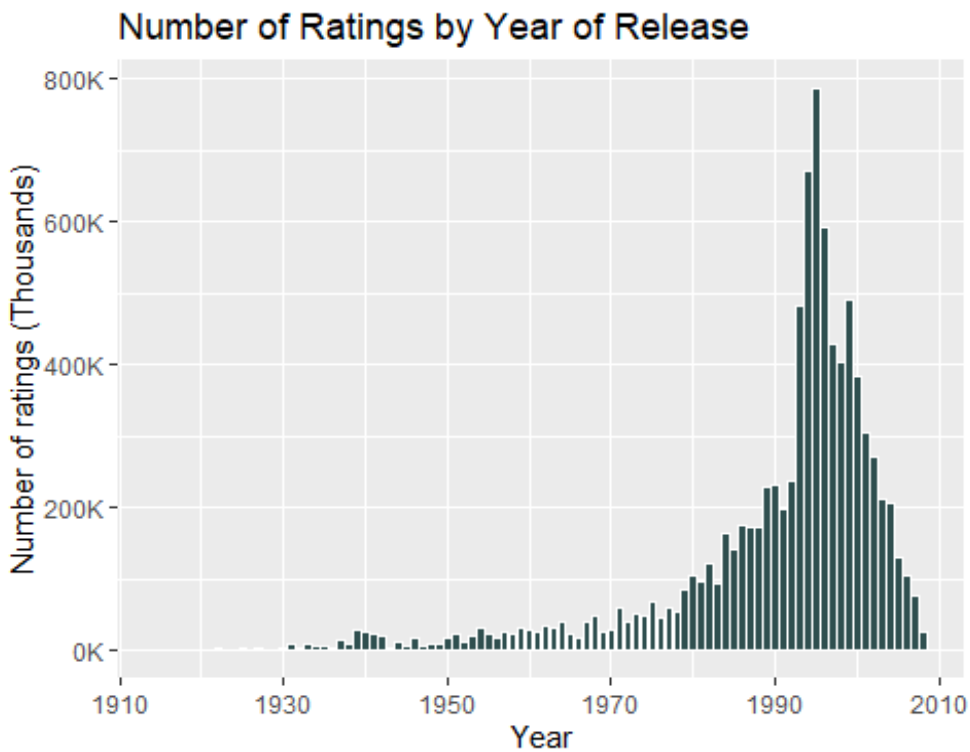
$$Y_{u,i} = \mu + b_i + b_u + b_g + \epsilon_{u,i}$$

```
# Adjusting the model for genre effect b_g
b_g <- train %>%
  left_join(b_i, by = "movieId") %>%
  left_join(b_u, by = "userId") %>%
  group_by(genre) %>%
  summarize(b_g = mean(rating - mu - b_i - b_u))

# RMSE based on mu, b_i, b_u and b_g
predicted_b_g <- test %>%
  left_join(b_i, by = 'movieId') %>%
  left_join(b_u, by = "userId") %>%
  left_join(b_g, by = "genre") %>%
  mutate(pred = mu + b_i + b_u + b_g) %>%
  pull(pred)

RMSE(test$rating, predicted_b_g)

## [1] 0.865414
```

The calculated RMSE $\mu + b_i + b_u + b_g$: 0.865414

To assess whether the release year influences ratings, we examine its effect within the dataset, which includes movies released between 1915 and 2008. The figure below shows that ratings are not evenly distributed across release years, indicating the presence of a release-year bias. Movies exhibit systematic differences by release period: older films may receive higher average ratings due to nostalgia effects, more recent releases often have fewer ratings and potentially lower averages, and certain decades may favor particular genres.

```
# Average rating per year of release in the edx dataset
edx_set %>%
  group_by(year) %>%
  summarize(n = n()) %>%
  arrange(year) %>%
  ggplot(aes(x = year, y = n)) +
  geom_col(fill = "darkslategrey", color = "white") +
  scale_y_continuous(labels = scales::label_number(scale = 1e-3, suffix = "K"
)) +
  labs(x = "Year", y = "Number of ratings (Thousands)", title = "Number of Ra
tings by Year of Release")
```



This effect is incorporated into the model to improve predictive accuracy and reduce RMSE.

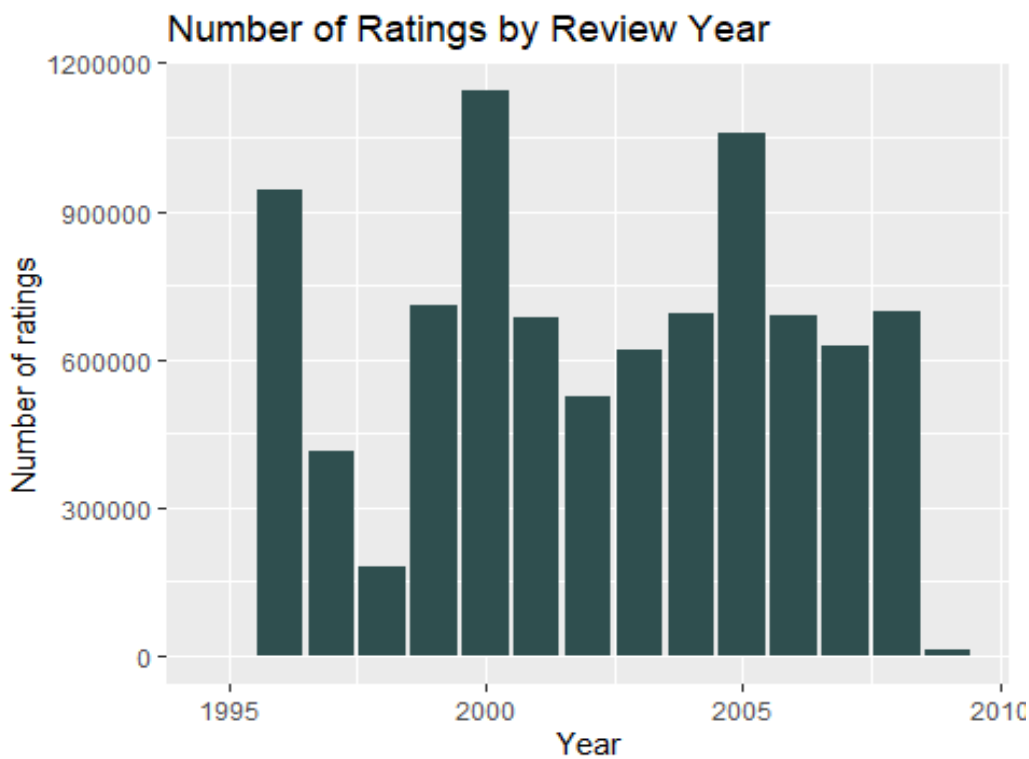
$$Y_{u,i} = \mu + b_i + b_u + b_g + b_y + \epsilon_{u,i}$$

```
# Adjusting the model for year of release effect b_y
b_y <- train %>%
  left_join(b_i, by = "movieId") %>%
  left_join(b_u, by = "userId") %>%
  left_join(b_g, by = "genre") %>%
  group_by(year) %>%
  summarize(b_y = mean(rating - mu - b_i - b_u - b_g))

# RMSE based on global mu, b_i, b_u, b_g and b_y effect

predicted_b_y <- test %>%
  left_join(b_i, by = "movieId") %>%
  left_join(b_u, by = "userId") %>%
  left_join(b_g, by = "genre") %>%
  left_join(b_y, by = "year") %>%
  mutate(pred = mu + b_i + b_u + b_g + b_y) %>%
  pull(pred)

RMSE(test$rating, predicted_b_y)

## [1] 0.8651204
```

The calculated RMSE $\mu + b_i + b_u + b_g + b_y$: 0.8651204

The MovieLens dataset includes a timestamp variable indicating when ratings were submitted. Although timestamps could be aggregated at finer levels (e.g. day or week), greater granularity increases estimation variance and the risk of overfitting without meaningfully improving predictive performance. Therefore, timestamps are aggregated at the yearly level to capture broader yearly trends.

```
# Distribution of ratings based on movie review year in the edx dataset
edx_set %>% group_by(review) %>%
  summarize(n = n()) %>%
  arrange(review) %>%
  ggplot(aes(x=review, y= n)) +
  geom_col(fill = "darkslategrey") +
  labs(x = "Year", y = "Number of ratings", title = "Number of Ratings by Rev
iew Year")
```



The systematic variation in ratings by review year, supporting the inclusion of a review-year bias term.

$$Y_{u,i} = \mu + b_i + b_u + b_g + b_y + b_r + \epsilon_{u,i}$$

```r
# Adjusting the model for review date effect (b_r)
b_r <- train %>%
  left_join(b_i, by = "movieId") %>%
  left_join(b_u, by  = "userId") %>%
  left_join(b_g, by = "genre") %>%
  left_join(b_y, by = "year") %>%
  group_by(review) %>%
  summarize(b_r = mean(rating - mu - b_i - b_u - b_g - b_y))

# RMSE based on mu, b_i, b_u, b_g, b_y and b_r effect
predicted_b_r <- test %>%
  left_join(b_i, by = "movieId") %>%
  left_join(b_u, by = "userId") %>%
  left_join(b_g, by = "genre") %>%
  left_join(b_y, by = "year") %>%
  left_join(b_r, by = "review") %>%
  mutate(pred = mu + b_i + b_u + b_g + b_y + b_r) %>%
  pull(pred)

RMSE(test$rating, predicted_b_r)

## [1] 0.8650206
```

Although the resulting improvement in RMSE is negligible, this effect is retained for completeness and consistency within the model.

The calculated RMSE $\mu + b_i + b_u + b_g + b_y + b_r$: 0.8650206

Exploratory analysis revealed systematic patterns across users, movies, genres, and time, indicating that rating behavior is influenced by consistent biases rather than random variation. These findings justify the use of a bias-based modeling approach that explicitly accounts for movie and user effects, genre differences, and time related dynamics.

Structural Challenges: Recommendation systems face inherent structural challenges, including data sparsity, cold start, and long-tail bias. Most users rate only a small subset of movies, limiting reliable preference estimation. Cold start occurs when new users or movies lack sufficient data for collaborative filtering. Additionally, ratings follow a long-tail distribution, where a small number of popular movies receive most interactions, potentially biasing predictions toward popularity. While regularization reduces overfitting and improves stability, it does not eliminate sparsity, solve true cold start problems, or remove popularity bias.

Regularization

Due to the high sparsity of the MovieLens dataset, many users and movies have only a limited number of ratings. Estimating effects using simple averages can therefore lead to overfitting, particularly for entities with small sample sizes. For example, ranking the “top 10” movies based on raw averages often yields obscure titles with very few ratings, as a

small number of high scores can inflate their mean. Such estimates are unstable and overly optimistic.

```
# 10 highest rated movies before regularization in the edx dataset
edx_set %>% group_by(movieId, title) %>%
  summarize(avg_rating = mean(rating),
            n_ratings = n()) %>%
  arrange(desc(avg_rating))
```

```
## # A tibble: 10,677 × 4
## # Groups:   movieId [10,677]
## movieId title                                          avg_rating n_ratings
## <int>   <chr>                                             <dbl>       <int>
##  3226 Hellhounds on My Trail                                  5           1
##  2   33264 Satan's Tango (Sátántangó)                         5           2
##  3   42783 Shadows of Forgotten Ancestors                     5           1
##  4   51209 Fighting Elegy (Kenka erejii)                      5           1
##  5   53355 Sun Alley (Sonnenallee)                            5           1
##  6   64275 Blue Light, The (Das Blaue Licht)                  5           1
##  7   5194 Who's Singin' Over There? (a.k.a. Who Sings Ove…  4.75         4
##  8   26048 Human Condition II, The (Ningen no joken II)     4.75         4
##  9   26073 Human Condition III, The (Ningen no joken III)   4.75         4
## 10   65001 Constantine's Sword                               4.75         2
## # ℹ 10,667 more rows
```

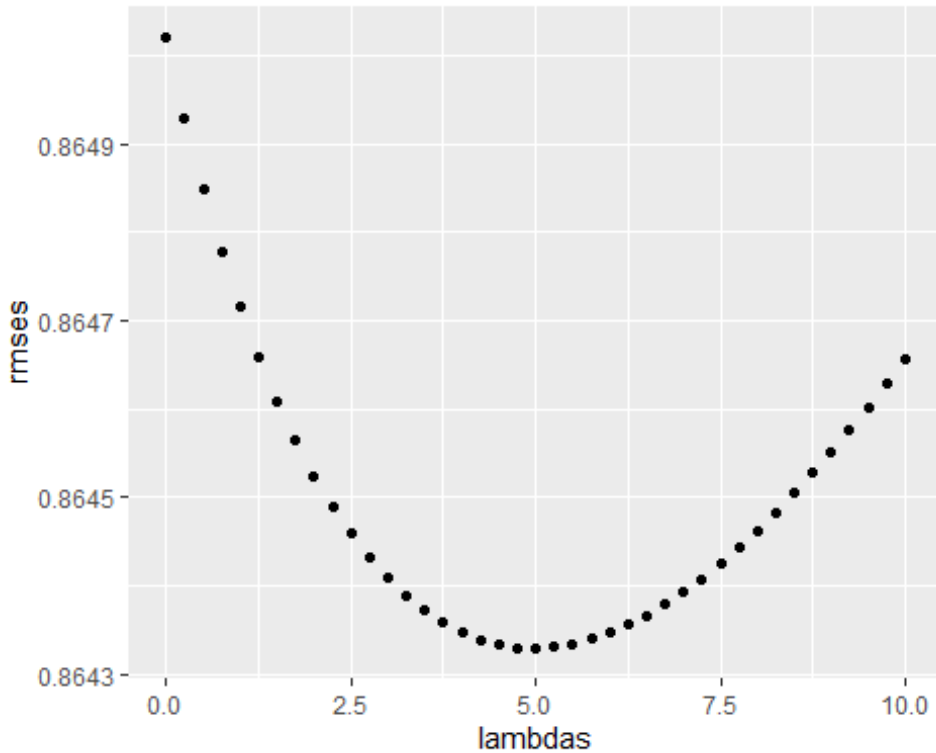
To address this issue, regularization was introduced through a penalty parameter, λ. Regularization shrinks extreme bias estimates toward zero when the number of observations is small, reducing variance and improving generalization, shrinking values toward the overall mean when uncertainty is high.

```
# Try values 0 - 10 in increment of 0.25 to determine the best lambda
lambdas <- seq(from=0, to=10, by=0.25)

# Regularise the model, predicting ratings and calculating RMSE for each valu
e of lambda in the range lambdas
rmses <- sapply(lambdas, function(l){

  # Calculate average rating across training data
  mu <- mean(train$rating)

  # Compute regularized movie bias term
  b_i <- train %>%
    group_by(movieId) %>%
    summarize(b_i = sum(rating - mu)/(n()+l))

  # Compute regularized user bias term
  b_u <- train %>%
    left_join(b_i, by="movieId") %>%
    group_by(userId) %>%
```

```r
    summarize(b_u = sum(rating - b_i - mu)/(n()+l))

  # Compute regularized genre bias term
  b_g <- train %>%
    left_join(b_i, by = "movieId") %>%
    left_join(b_u, by = "userId") %>%
    group_by(genre) %>%
    summarize(b_g = sum(rating - mu - b_i - b_u)/ (n() + l))

  # Compute regularized year bias term
  b_y <- train %>%
    left_join(b_i, by = "movieId") %>%
    left_join(b_u, by = "userId") %>%
    left_join(b_g, by = "genre") %>%
    group_by(year) %>%
    summarize(b_y = sum(rating - mu - b_i - b_u - b_g) / (n() +l))

  # Compute regularized review year term
  b_r <- train %>%
    left_join(b_i, by = "movieId") %>%
    left_join(b_u, by = "userId") %>%
    left_join(b_g, by = "genre") %>%
    left_join(b_y, by = "year") %>%
    group_by(review) %>%
    summarize(b_r = sum(rating - mu - b_i - b_u - b_g - b_y) / (n() + l))

  # Compute predictions on test set based on these above terms
  predicted_ratings <- test %>%
    left_join(b_i, by = "movieId") %>%
    left_join(b_u, by = "userId") %>%
    left_join(b_g, by = "genre") %>%
    left_join(b_y, by = "year") %>%
    left_join(b_r, by = "review") %>%
    mutate(pred = mu + b_i + b_u + b_g + b_y + b_r) %>%
    pull(pred)

  # Output RMSE of these predictions
  return(RMSE(predicted_ratings, test$rating))
})

# Find best lambda
lambda <- lambdas[which.min(rmses)]
lambda

## [1] 5
```

Based on RMSE evaluations across range of lambda values, the optimal regularization parameter that is associated with the lowest RMSE values was determined to be $\lambda = 5$.

```r
# Lambdas vs RMSE
qplot(lambdas, rmses)
```



Lambdas vs RMSE plot

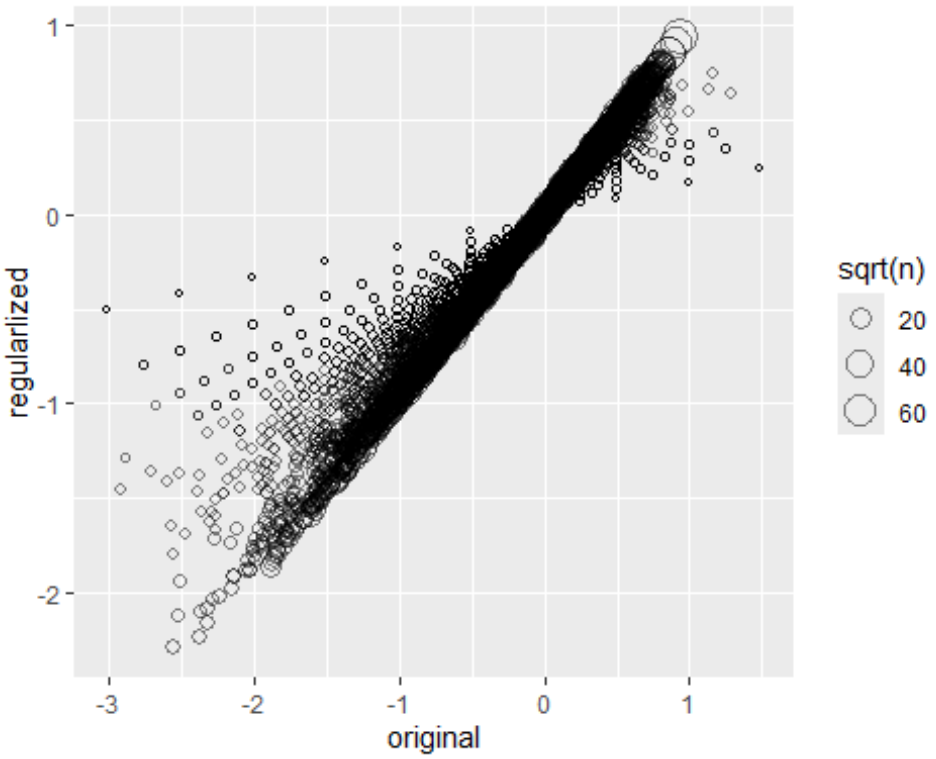
With regularization, movies and users with few ratings are pulled more strongly toward the global mean, resulting in more stable and reliable predictions. After applying regularization, the list of the 10 highest-rated movies appears more realistic, as the titles consist primarily of well-known and critically acclaimed films rather than obscure movies with only a handful of ratings.

```r
# 10 Highest rated movies after regularization in the edx set
edx_set %>%
  group_by(movieId, title) %>%
  summarize(
    n_ratings = n(),
    b_i = sum(rating - mu) / (n() + lambda),
    reg_rating = mu + b_i,
    .groups = "drop") %>%
  arrange(desc(reg_rating)) %>%
  slice_head(n = 10)
```

```
## # A tibble: 10 × 5
##    movieId title                                n_ratings  b_i    r_rating
##      <int> <chr>                                  <int>   <dbl>    <dbl>
##  1     318 Shawshank Redemption, The              28015   0.942    4.45
##  2     858 Godfather, The                         17747    0.903   4.42
##  3      50 Usual Suspects, The                    21648    0.853   4.37
##  4     527 Schindler's List                       23193    0.851   4.36
##  5     912 Casablanca                              11232   0.808    4.32
##  6     904 Rear Window                             7935    0.806    4.32
##  7     922 Sunset Blvd. (a.k.a. Sunset Boulevard)  2922    0.802    4.31
##  8    1212 Third Man, The                          2967    0.798    4.31
##  9    3435 Double Indemnity                        2154    0.797    4.31
## 10    1178 Paths of Glory                          1571    0.794    4.31
```

In large and sparse datasets such as MovieLens, many movies and users have only a limited number of ratings, and certain genres, release years, or review periods may also contain relatively few observations. This imbalance in the data can produce unstable and inflated bias estimates, particularly for entities with small sample sizes. Regularization mitigates this issue by introducing a penalty term that shrinks parameter estimates toward zero, thereby reducing overfitting and improving the model's ability to generalize to validation data. Consequently, movies and users with few ratings are pulled more strongly toward the global mean, resulting in more stable, realistic, and reliable predictions.

```r
# Effect of regularization on test set
mu <- mean(test$rating)
avgs <- test %>%
  group_by(movieId) %>%
  summarize(b_i = mean(rating - mu))

reg_avgs <- test %>%
  group_by(movieId) %>%
  summarize(
    b_i = sum(rating - mu) / (n() + lambda),
    n_i = n())

tibble(original = avgs$b_i,
regularlized = reg_avgs$b_i,
n = reg_avgs$n_i) %>%
ggplot(aes(original, regularlized, size=sqrt(n))) +
geom_point(shape=1, alpha=0.5)
```



The effect of regularization

Model performance evaluation showed that movie bias and user bias contributed the largest reductions in RMSE, while genre, release year, and review year effects provided smaller incremental improvements.

Factorization

As some users prefer certain genres or may rate higher or lower than average, the matrix factorization model can observe these trends. The table shows the correlations between the ratings of five selected movies across users. Each value represents how similarly users rated two movies. A positive correlation close to 1 indicates that users who liked one movie also tended to like the other, while low correlation values indicate that users who liked one movie tended to dislike the other and that these movies attract different audiences. The high correlation between “Pride and Prejudice” and “Sense and Sensibility,” as well as between the “Godfather” films, suggests that each type of movie appeals to specific audiences. A value of “1” indicates that a movie perfectly correlates with itself.

```r
# Movie ratings correlation - latent factors
# Count number of ratings per movie and per user
movie_counts <- edx %>% count(movieId)
user_counts  <- edx %>% count(userId)

# Filter edx set for movies with more than 600 ratings and user who rated mor
e than 600 movies
train_small <- edx %>%
  semi_join(movie_counts %>% filter(n >= 600), by = "movieId") %>%
  semi_join(user_counts  %>% filter(n >= 600), by = "userId")

# Title in the dataset
available_titles <- sort(unique(train_small$title))

# Select popular movies for correlation example
movies_for_table <- c(
 "Godfather, The (1972)",
 "Godfather: Part II, The (1974)",
 "Goodfellas (1990)",
 "Pride & Prejudice (2005)",
 "Sense and Sensibility (1995)",
 "Casino (1995)")

# Prepare wide matrix of ratings
ratings_wide <- train_small %>%
  filter(title %in% movies_for_table) %>%
  select(userId, title, rating) %>%
  pivot_wider(names_from = title, values_from = rating) %>%
  column_to_rownames(var = "userId")

# Determine the degree of correlations
cor_table <- cor(ratings_wide, use = "pairwise.complete.obs") %>%
  round(3) %>%
  as.data.frame()

if(require(knitr)){
  knitr::kable(cor_table)
} else {
  cor_table}
```

	Casino (1995)	Sense and Sensibility (1995)	Godfather, The (1972)	Godfather: Part II, The (1974)	Goodfellas (1990)	Pride & Prejudice (2005)
Casino (1995)	1.000	0.077	0.307	0.340	0.408	0.122
Sense and Sensibility (1995)	0.077	1.000	0.149	0.168	0.159	0.460
Godfather, The (1972)	0.307	0.149	1.000	0.778	0.417	0.094
Godfather: Part II, The (1974)	0.340	0.168	0.778	1.000	0.468	0.139
Goodfellas (1990)	0.408	0.159	0.417	0.468	1.000	0.078
Pride & Prejudice (2005)	0.122	0.460	0.094	0.139	0.078	1.000

Final Models and Results

Regularized bias model

After refining the model using train (80%) and test (20%) splits of the edx dataset, the final model was trained on the full edx dataset and applied to the validation set. The model incorporated movie, user, genre, release year, and review year effects, regularized using the optimal $\lambda = 5$.

```
# Using full edx dataset to model effect of bias and regularize with the best
lambda

# With movie bias effect
b_i <- edx_set %>%
  group_by(movieId) %>%
  summarize(b_i = sum(rating - mu) / (n() + lambda))

# With user bias effect
b_u <- edx_set %>%
  left_join(b_i, by = 'movieId') %>%
  group_by(userId) %>%
  summarize(b_u = sum(rating - mu - b_i) / (n() + lambda))
```

```r
# With genre bias effect
b_g <- edx_set %>%
  left_join(b_i, by = "movieId") %>%
  left_join(b_u, by = "userId") %>%
  group_by(genre) %>%
  summarize(b_g = sum(rating - mu - b_i - b_u) / (n() + lambda))


# With year bias effect
b_y <- edx_set %>%
  left_join(b_i, by = "movieId") %>%
  left_join(b_u, by = "userId") %>%
  left_join(b_g, by = "genre") %>%
  group_by(year) %>%
  summarize(b_y = sum(rating - mu - b_i - b_u - b_g) / (n() + lambda))

# With review bias effect
b_r <- edx_set %>%
  left_join(b_i, by = "movieId") %>%
  left_join(b_u, by = "userId") %>%
  left_join(b_g, by = "genre") %>%
  left_join(b_y, by = "year") %>%
  group_by(review) %>%
  summarize(b_r = sum(rating - mu - b_i - b_u - b_g - b_y) / (n() + lambda))

# Predict ratings for validation set
predicted_validation <- validation_set %>%
  left_join(b_i, by = "movieId") %>%
  left_join(b_u, by = "userId") %>%
  left_join(b_g, by = "genre") %>%
  left_join(b_y, by = "year") %>%
  left_join(b_r, by = "review") %>%
  mutate(pred = mu + b_i + b_u + b_g + b_y + b_r) %>%
  pull(pred)

# Calculate best RMSE for validation set
RMSE(validation_set$rating, predicted_validation)

## [1] 0.8643227
```

The final RMSE achieved on the validation set: 0.8643227

This result meets the target threshold (0.86490) and demonstrates that a regularized additive bias model can effectively capture systematic variation in sparse recommendation data while maintaining strong generalization performance.

Matrix factorization model

The matrix model takes into account latent factors of user preferences and behaviors and product characteristics to predict ratings for similar items based on these clues.

```
library(recosystem)

  # Set a reproducible seed
set.seed(1, sample.kind = "Rounding")

  # Convert train_set and validation_set to recosystem format
train_data <- with(edx, data_memory(
  user_index = userId,
  item_index = movieId,
  rating = rating
))

test_data <- with(final_holdout_test, data_memory(
  user_index = userId,
  item_index = movieId,
  rating = rating
))

  # Create the Recosystem model
r <- Reco()

  # Tune hyperparameters on train_set using cross-validation
opts <- r$tune(
  train_data,
  opts = list(
    dim = c(10, 20, 30),       # latent factor dimensions
    costp_l2 = c(0.01, 0.1),   # regularization for user factors
    costq_l2 = c(0.01, 0.1),   # regularization for item factors
    costp_l1 = 0, costq_l1 = 0,
    lrate = c(0.01, 0.1),      # learning rates
    nthread = 1,
    niter = 10,
    verbose = FALSE))

  # Train the model using the best tuning parameters
r$train(
  train_data,
  opts = c(opts$min, nthread = 1, niter = 100, verbose = FALSE))

  # Predict ratings for validation_set only
reco_pred <- r$predict(test_data, out_memory())

  # Calculate RMSE on validation_set
```

```
validation_rmse <- RMSE(final_holdout_test$rating, reco_pred)
validation_rmse

## [1] 0.7818844
```

The matrix factorization model has achieved far better results in predicting ratings for movies than regularized bias model, by achieving RMSE of 0.7818844. This result shows that by considering latent factors and specific movie/user interaction the model was able to train better in predicting ratings that similar users would have given to similar movies.

Conclusion

The aim of this project was to develop an efficient recommendation system using the MovieLens dataset. Starting from a model with the global average rating, performance was progressively improved by incorporating movie, user, genre, release year, and review year effects. Movie effects contributed substantially, while user effects were even more influential, and the other biases provided modest improvements. Regularization was applied to shrink extreme estimates for users and movies with few ratings, reducing variance and improving generalization. The optimal λ of 5 was selected, and the final model achieved strong predictive accuracy on the fianl-holdout-test set. The base model using the global mean produced an RMSE of 1.060331. The regularized bias model achieved the RMSE of 0.8643227 on the validation set, which is lower than the target value of 0.86490.The matrix factorization model was built by considering both movie and user biases, accounting for latent factors, and achieved a significantly better RMSE of 0.7818844. Matrix factorization compresses the large user–movie matrix, allowing the model to make good predictions even when many ratings are missing. Matrix factorization performs better because it models biases, learns hidden preference patterns, and captures interactions between users and movies, leading to more accurate rating predictions.

Comparison of RMSE Across Recommendation Models

Model	RMSE
RMSE Target	0.86490
Global Mean Model	1.06033
Regularized Bias Model	0.86432
Matrix Factorization	0.78188

Other models for predicting movie ratings such as KNN and bootstrapping exist, but they will require more resources than the average computer can provide.

References: McKinsey. (2023). What is personalization? Retrieved February 2026, from https://www.mckinsey.com/featured-insights/mckinsey-explainers/what-is-personalization